

Problem Statement 3 – Unit Testing FastAPI Product Management API

Scenario

A FastAPI backend is built for a **Product Management System** used by an e-commerce application.

API Endpoints

- GET /products – Fetch all products
- GET /products/{id} – Fetch product by ID
- POST /products – Add a new product
- PUT /products/{id} – Update product details
- DELETE /products/{id} – Delete product

Product Model

- id
- name
- price
- category
- stock

Testing Task

Write **pytest unit tests** to check:

1. Fetching all products returns status 200
2. Fetching non-existing product returns 404
3. Product creation validates required fields
4. Product update modifies correct fields
5. Product deletion removes product successfully

Goal of Testing

- Ensure correct CRUD behavior
- Validate error handling
- Test response structure and status codes

Problem Statement 4 – Unit Testing React Product Create & Delete Flow

Scenario

The React app allows users to **add and delete products** using the FastAPI backend.

UI Features

- Form to add a product (POST /products)
- Delete button for each product (DELETE /products/{id})
- Success and error messages

Testing Task

Write Jest tests to verify:

1. Form fields accept user input correctly
2. Submit button triggers POST API call
3. Success message appears on successful product creation
4. Delete button triggers DELETE API call
5. Product is removed from UI after deletion
6. Error message appears on API failure

Goal of Testing

- Validate form handling
- Ensure correct API calls
- Verify UI updates after backend operations